

-- Week 2 Task: E-Commerce Sales Database

CREATE DATABASE shopease;

USE shopease;

-- Table1 : Customers

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    state VARCHAR(50) NOT NULL,  
    join_date DATE NOT NULL,  
    is_premium BOOLEAN DEFAULT FALSE  
);
```

-- Index for filtering by city/state

CREATE INDEX idx_customers_city ON customers(city);

CREATE INDEX idx_customers_state ON customers(state);

-- Table2 : Products

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100) NOT NULL,  
    category VARCHAR(50) NOT NULL,
```

```
brand    VARCHAR(50) NOT NULL,  
unit_price  DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),  
stock_qty   INT      NOT NULL DEFAULT 0 CHECK (stock_qty >= 0)  
);
```

-- Index for filtering by category

```
CREATE INDEX idx_products_category ON products(category);
```

-- Table3 : Orders

```
CREATE TABLE orders (  
    order_id    INT      PRIMARY KEY,  
    customer_id INT      NOT NULL,  
    order_date  DATE     NOT NULL,  
    status      VARCHAR(20) NOT NULL DEFAULT 'Pending'  
        CHECK (status IN ('Pending','Shipped','Delivered','Cancelled')),  
    total_amount DECIMAL(12,2) NOT NULL CHECK (total_amount >= 0),  
  
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

-- Index for date-based filtering and sorting

```
CREATE INDEX idx_orders_date ON orders(order_date);
```

```
CREATE INDEX idx_orders_status ON orders(status);
```

-- Table4 : order_items

```
CREATE TABLE order_items (  
    order_id    INT      PRIMARY KEY,  
    product_id  INT      NOT NULL,  
    quantity    INT      NOT NULL DEFAULT 1,  
    unit_price  DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),  
    total_amount DECIMAL(12,2) NOT NULL CHECK (total_amount >= 0),  
    FOREIGN KEY (order_id) REFERENCES orders(order_id),  
    FOREIGN KEY (product_id) REFERENCES products(product_id)
```

```

item_id    INT        PRIMARY KEY,
order_id   INT        NOT NULL,
product_id INT        NOT NULL,
quantity   INT        NOT NULL CHECK (quantity > 0),
unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),
discount_pct DECIMAL(5,2) DEFAULT 0 CHECK (discount_pct BETWEEN 0 AND 100),

FOREIGN KEY (order_id) REFERENCES orders(order_id),
FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- ===== INSERT: customers =====

INSERT INTO customers VALUES
(101, 'Aarav', 'Sharma', 'aarav.s@email.com', 'Mumbai', 'Maharashtra', '2024-01-15', TRUE),
(102, 'Priya', 'Patel', 'priya.p@email.com', 'Ahmedabad', 'Gujarat', '2024-02-20', FALSE),
(103, 'Rohan', 'Gupta', 'rohan.g@email.com', 'Delhi', 'Delhi', '2024-03-10', TRUE),
(104, 'Sneha', 'Reddy', 'sneha.r@email.com', 'Hyderabad', 'Telangana', '2024-04-05', FALSE),
(105, 'Vikram', 'Singh', 'vikram.s@email.com', 'Jaipur', 'Rajasthan', '2024-05-12', TRUE),
(106, 'Ananya', 'Iyer', 'ananya.i@email.com', 'Chennai', 'Tamil Nadu', '2024-06-18', FALSE),
(107, 'Karan', 'Mehta', 'karan.m@email.com', 'Pune', 'Maharashtra', '2024-07-22', TRUE),
(108, 'Divya', 'Nair', 'divya.n@email.com', 'Kochi', 'Kerala', '2024-08-30', FALSE);

-- ===== INSERT: products =====

INSERT INTO products VALUES
(201, 'Wireless Earbuds', 'Electronics', 'BoAt', 1499.00, 250),

```

```
(202, 'Cotton T-Shirt', 'Clothing', 'Levis', 799.00, 500),
(203, 'Smart Watch', 'Electronics', 'Noise', 2999.00, 150),
(204, 'Running Shoes', 'Clothing', 'Nike', 4599.00, 120),
(205, 'Bluetooth Speaker', 'Electronics', 'JBL', 3499.00, 200),
(206, 'Bedsheet Set', 'Home', 'Spaces', 1299.00, 300),
(207, 'Laptop Stand', 'Electronics', 'AmazonBasics', 899.00, 180),
(208, 'Cushion Covers (Set)', 'Home', 'HomeCenter', 599.00, 400);
```

```
-- ===== INSERT: orders =====
```

```
INSERT INTO orders VALUES
```

```
(1001, 101, '2024-08-01', 'Delivered', 4498.00),
(1002, 102, '2024-08-03', 'Delivered', 799.00),
(1003, 103, '2024-08-05', 'Shipped', 7498.00),
(1004, 101, '2024-08-10', 'Delivered', 3499.00),
(1005, 104, '2024-08-12', 'Cancelled', 2999.00),
(1006, 105, '2024-08-15', 'Delivered', 5898.00),
(1007, 106, '2024-08-18', 'Pending', 1299.00),
(1008, 103, '2024-08-20', 'Delivered', 899.00),
(1009, 107, '2024-08-25', 'Shipped', 6098.00),
(1010, 108, '2024-08-28', 'Delivered', 1598.00);
```

```
-- ===== INSERT: order_items =====
```

```
INSERT INTO order_items VALUES
```

```
(5001, 1001, 201, 2, 1499.00, 0),
(5002, 1001, 207, 1, 899.00, 10),
(5003, 1002, 202, 1, 799.00, 0),
```

```

(5004, 1003, 203, 1, 2999.00, 0),
(5005, 1003, 204, 1, 4599.00, 5),
(5006, 1004, 205, 1, 3499.00, 0),
(5007, 1005, 203, 1, 2999.00, 0),
(5008, 1006, 201, 1, 1499.00, 10),
(5009, 1006, 204, 1, 4599.00, 5),
(5010, 1007, 206, 1, 1299.00, 0),
(5011, 1008, 207, 1, 899.00, 0),
(5012, 1009, 205, 1, 3499.00, 0),
(5013, 1009, 208, 2, 599.00, 15),
(5014, 1010, 206, 1, 1299.00, 0),
(5015, 1010, 208, 1, 599.00, 0);

```

```
SELECT * FROM CUSTOMERS ;
```

Result Grid   Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 								
	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
SELECT * FROM ORDER_ITEMS;
```

Result Grid						
Filter Rows:						
	item_id	order_id	product_id	quantity	unit_price	discount_pct
▶	5001	1001	201	2	1499.00	0.00
	5002	1001	207	1	899.00	10.00
	5003	1002	202	1	799.00	0.00
	5004	1003	203	1	2999.00	0.00
	5005	1003	204	1	4599.00	5.00
	5006	1004	205	1	3499.00	0.00
	5007	1005	203	1	2999.00	0.00
	5008	1006	201	1	1499.00	10.00
	5009	1006	204	1	4599.00	5.00
	5010	1007	206	1	1299.00	0.00
	5011	1008	207	1	899.00	0.00
	5012	1009	205	1	3499.00	0.00
	5013	1009	208	2	599.00	15.00
	5014	1010	206	1	1299.00	0.00
	5015	1010	208	1	599.00	0.00
	5020	1011	208	1	999.00	0.00
	5021	1011	202	1	599.00	0.00
▲	NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM ORDERS ;

Result Grid					
Filter Rows:					
	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1003	103	2024-08-05	Shipped	7498.00
	1004	101	2024-08-10	Delivered	3499.00
	1005	104	2024-08-12	Cancelled	2999.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00
	1010	108	2024-08-28	Delivered	1598.00
	1011	102	2026-06-29	Pending	1598.00
●	NULL	NULL	NULL	NULL	1598.00

SELECT * FROM PRODUCTS;

Result Grid						
		Filter Rows:			Edit:	Export/Import:
	product_id	product_name	category	brand	unit_price	stock_qty
▶	201	Wireless Earbuds	Electronics	BoAt	1499.00	249
	202	Cotton T-Shirt	Clothing	Levis	799.00	499
	203	Smart Watch	Electronics	Noise	2999.00	150
	204	Running Shoes	Clothing	Nike	4599.00	120
	205	Bluetooth Speaker	Electronics	JBL	3499.00	200
	206	Bedsheet Set	Home	Spaces	1299.00	300
	207	Laptop Stand	Electronics	AmazonBasics	899.00	180
	208	Cushion Covers (Set)	Home	HomeCenter	599.00	398
*	NULL	NULL	NULL	NULL	NULL	NULL

/* Section A — SQL Basics (SELECT, Constraints, Primary Keys)

These questions test your understanding of basic data retrieval, table structure, and database constraints.

*/

-- Q1. Write a query to display all columns and rows from the customer's table.

SELECT *

FROM CUSTOMERS;

Result Grid								
Filter Rows:								
Edit:								
Export/Import:								
Wrap Cell Content:								
	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

-- Insight:

-- Above query retrieves the complete dataset from the customers table.

-- Q2. Retrieve only the first_name, last_name, and city of all customers.

SELECT FIRST_NAME, LAST_NAME, CITY

FROM CUSTOMERS;

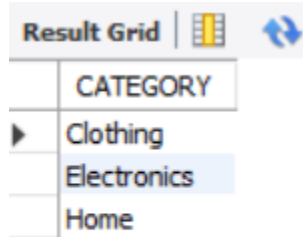
Result Grid			
Filter Rows:			
E			
	FIRST_NAME	LAST_NAME	CITY
▶	Aarav	Sharma	Mumbai
	Priya	Patel	Ahmedabad
	Rohan	Gupta	Delhi
	Sneha	Reddy	Hyderabad
	Vikram	Singh	Jaipur
	Ananya	Iyer	Chennai
	Karan	Mehta	Pune
	Divya	Nair	Kochi

-- Insight:

-- Above query selects only specific columns (FIRST_NAME, LAST_NAME, CITY) to focus on customer identity and location.

-- Q3. List all unique categories available in the products table.

```
SELECT DISTINCT CATEGORY  
FROM PRODUCTS ;
```



CATEGORY
Clothing
Electronics
Home

-- Insight:

-- DISTINCT allow only unique product categories are displayed.

/*Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Primary Keys in each table:

customers -> customer_id

products -> product_id

orders -> order_id

order_items -> item_id

Above are the primary keys for tables present in shopease database.

Explanation

A Primary Key is :

Uniquely identifies each record in a table , Cannot be NULL , Cannot have duplicate values

Why primary key is important:

It Ensures data integrity , helps in establishing relationships (Foreign Keys) , allows efficient data retrieval.

*/

/* Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

Constraints applied for email column :-

email VARCHAR(100) UNIQUE NOT NULL

(Unique, not null)

Unique - No 2 or more than 2 customers can have the same email.

Not null - Email cannot be empty. (Mandatory field).

What happens if duplicate is inserted?

SQL will throw an error like Duplicate entry 'abc@gmail.com' for key 'email'.

*/

-- Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

INSERT INTO products VALUES

(210, 'Laptop Cover', 'Electronics', 'BoAt', -50, 25)

/* It will give error after running this query.

Reason - We use constraint CHECK (unit_price > 0)

Error Explanation :-

The database rejects the insertion because unit_price is negative, violating the CHECK constraint.

Constraint ensures that only valid (positive) prices are stored in the table.

*/

/* Section B — Filtering & Optimization (WHERE, Indexes)

These questions test your ability to filter data effectively and understand how indexes improve query performance.

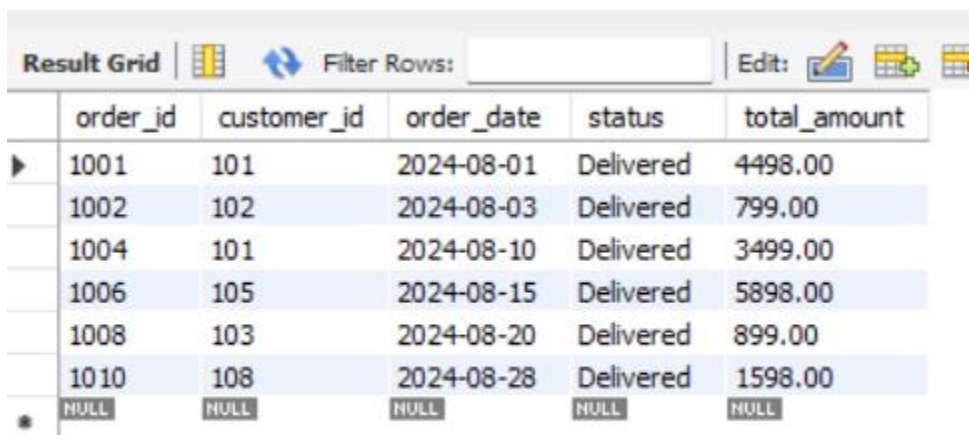
*/

-- Q7. Retrieve all orders with status ='Delivered'

SELECT *

FROM ORDERS

WHERE STATUS = 'Delivered' ;



The screenshot shows a database query result grid. The grid has a toolbar at the top with icons for 'Result Grid', 'Filter Rows', and 'Edit'. The table has six columns: order_id, customer_id, order_date, status, and total_amount. The data is as follows:

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00
NULL	NULL	NULL	NULL	NULL

-- Insight:

-- Filters only successfully completed (delivered) orders for analysis using where clause.

-- Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

SELECT *

FROM PRODUCTS

WHERE CATEGORY = 'Electronics' AND UNIT_PRICE > 2000 ;

Result Grid Filter Rows: Edit: Export/Import:						
	product_id	product_name	category	brand	unit_price	stock_qty
▶	203	Smart Watch	Electronics	Noise	2999.00	150
	205	Bluetooth Speaker	Electronics	JBL	3499.00	200
✱	NULL	NULL	NULL	NULL	NULL	NULL

-- Insight:

-- Helps identify high value electronics products.

-- Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

SELECT *

FROM CUSTOMERS

WHERE YEAR(JOIN_DATE) = 2024 AND STATE = 'Maharashtra';

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:								
	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

-- Insight:

-- Filters recent customers from a specific region Maharashtra.

-- Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

SELECT *

FROM ORDERS

WHERE ORDER_DATE BETWEEN '2024-08-10' AND '2024-08-25' AND STATUS <> 'cancelled' ;

Result Grid					
	order_id	customer_id	order_date	status	total_amount
▶	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00
⌵	NULL	NULL	NULL	NULL	NULL

-- Insight:

-- Filters valid orders within a given time range for trend analysis.

/* Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

What index idx_orders_date does:

```
CREATE INDEX idx_orders_date ON orders(order_date);
```

This index will -

Creates a sorted structure on order_date , Helps database find rows faster when filtering/sorting by date

Performance Benefit:=

Without index:

Full table scan (slow)

With index:

Direct lookup (fast)

Sample query using this index :

Query benefiting from index

```
SELECT *
```

```
FROM orders
```

WHERE order_date = '2024-08-15';

Explanation :-

The index on order_date improves performance by reducing number of rows scanned. Instead of scanning the entire table, the database uses index to directly locate matching records.

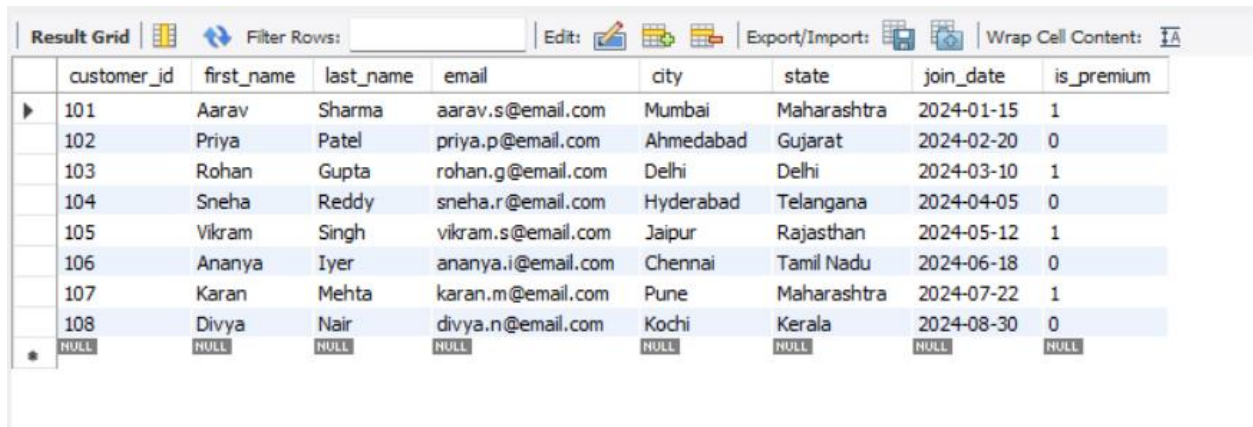
*/

-- Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

SELECT *

FROM CUSTOMERS

WHERE YEAR(JOIN_DATE) = 2024;



The screenshot shows a database query result grid with the following columns: customer_id, first_name, last_name, email, city, state, join_date, and is_premium. The data is as follows:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

/* Will index be used?

NO , Why

Because:

YEAR(join_date) applies a function on the column

Index is on raw column, not computed value

So Database cannot use index efficiently

SARGable Query (Index-friendly query)

SELECT *

FROM customers

WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31';

Explanation :-

A query is SARGable when it allows the database to use indexes efficiently.

Using functions like YEAR() on indexed columns prevents index usage.

Rewriting the query using a date range allows the index to be used, improving performance.

*/

/*Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

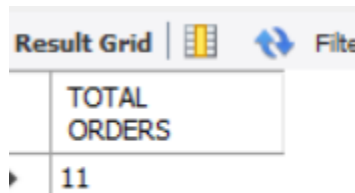
These questions test your ability to summarize and aggregate data.

*/

-- Q13. Count the total number of orders in the orders table.

SELECT COUNT(*) 'TOTAL ORDERS'

FROM ORDERS;



The screenshot shows a database interface with a 'Result Grid' tab. The grid contains a single column labeled 'TOTAL ORDERS' and a single row with the value '11'. There are icons for 'Filter' and 'Refresh' at the top of the grid.

TOTAL ORDERS
11

-- Insight:

-- Counts total records in orders table to understand business volume.

-- Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

SELECT SUM(TOTAL_AMOUNT) 'SUM of total_amount'

FROM ORDERS

WHERE STATUS = 'Delivered' ;

Result Grid		Filter Rows:
	SUM of total_amount	
▶	17191.00	

-- Insight:

-- Calculates actual earned revenue (only completed orders).

-- Q15. Calculate the average unit_price of products in each category.

```
SELECT CATEGORY , AVG(UNIT_PRICE) 'AVG PRICE'
FROM PRODUCTS
GROUP BY CATEGORY ;
```

Result Grid			Filter Rows:
	CATEGORY	AVG PRICE	
▶	Clothing	2699.000000	
	Electronics	2224.000000	
	Home	949.000000	949.000000

-- Insight:

-- Helps to compare pricing trends across different product categories.

-- Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

```
SELECT STATUS, COUNT(*) 'TOTAL ORDERS' , SUM(TOTAL_AMOUNT) 'TOTAL REVENUE'
FROM ORDERS
GROUP BY STATUS
```


ORDER BY 'TOTAL REVENUE' DESC ;

Result Grid  Filter Rows: Exp			
	STATUS	TOTAL ORDERS	TOTAL REVENUE
▶	Cancelled	1	2999.00
	Delivered	6	17191.00
	Pending	2	2897.00
	Shipped	2	13596.00

-- Insight:

-- Shows which order status contributes most to the revenue.

-- Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

SELECT CATEGORY, MAX(UNIT_PRICE) 'MOST EXPENSIVE' , MIN(UNIT_PRICE) 'CHEAPEST'

FROM PRODUCTS

GROUP BY CATEGORY;

Result Grid  Filter Rows: Exp			
	CATEGORY	MOST EXPENSIVE	CHEAPEST
▶	Clothing	4599.00	799.00
	Electronics	3499.00	899.00
	Home	1299.00	599.00

-- Insight:

-- Identifies price range within each category.

-- Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

SELECT CATEGORY , AVG(UNIT_PRICE) 'AVG PRICE'

FROM PRODUCTS

GROUP BY CATEGORY

HAVING AVG(UNIT_PRICE) > 2000 ;

Result Grid		Filter Rows:
	CATEGORY	AVG PRICE
▶	Clothing	2699.000000
	Electronics	2224.000000

/*Section D — Joins & Relationships

These questions test your ability to combine data from multiple tables using different types of JOINS.

*/

-- Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

SELECT O.ORDER_ID, O.ORDER_DATE, C.FIRST_NAME, C.LAST_NAME , O.TOTAL_AMOUNT

FROM ORDERS O INNER JOIN CUSTOMERS C

ON O.CUSTOMER_ID = C.CUSTOMER_ID ;

Result Grid					
		Filter Rows:			
		Export:			
		Wrap Cell Content:			
	ORDER_ID	ORDER_DATE	FIRST_NAME	LAST_NAME	TOTAL_AMOUNT
▶	1001	2024-08-01	Aarav	Sharma	4498.00
	1004	2024-08-10	Aarav	Sharma	3499.00
	1002	2024-08-03	Priya	Patel	799.00
	1011	2026-06-29	Priya	Patel	1598.00
	1003	2024-08-05	Rohan	Gupta	7498.00
	1008	2024-08-20	Rohan	Gupta	899.00
	1005	2024-08-12	Sneha	Reddy	2999.00
	1006	2024-08-15	Vikram	Singh	5898.00
	1007	2024-08-18	Ananya	Iyer	1299.00
	1009	2024-08-25	Karan	Mehta	6098.00
	1010	2024-08-28	Divya	Nair	1598.00

-- Insight:

-- Shows only those orders where a matching customer exists.

-- Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

```
SELECT C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME, O.ORDER_ID, O.ORDER_DATE,
O.TOTAL_AMOUNT
```

```
FROM CUSTOMERS C LEFT JOIN ORDERS O
```

```
ON C.CUSTOMER_ID = O.CUSTOMER_ID ;
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 						
	CUSTOMER_ID	FIRST_NAME	LAST_NAME	ORDER_ID	ORDER_DATE	TOTAL_AMOUNT
▶	101	Aarav	Sharma	1001	2024-08-01	4498.00
	101	Aarav	Sharma	1004	2024-08-10	3499.00
	102	Priya	Patel	1002	2024-08-03	799.00
	102	Priya	Patel	1011	2026-06-29	1598.00
	103	Rohan	Gupta	1003	2024-08-05	7498.00
	103	Rohan	Gupta	1008	2024-08-20	899.00
	104	Sneha	Reddy	1005	2024-08-12	2999.00
	105	Vikram	Singh	1006	2024-08-15	5898.00
	106	Ananya	Iyer	1007	2024-08-18	1299.00
	107	Karan	Mehta	1009	2024-08-25	6098.00
	108	Divya	Nair	1010	2024-08-28	1598.00

-- Insight:

-- Shows all customers, even those who never placed an order.

-- Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

```

SELECT O.ORDER_ID,
       P.PRODUCT_NAME,
       OI.QUANTITY,
       OI.UNIT_PRICE,
       OI.DISCOUNT_PCT
FROM ORDERS O
INNER JOIN ORDER_ITEMS OI
ON O.ORDER_ID = OI.ORDER_ID
INNER JOIN PRODUCTS P
ON OI.PRODUCT_ID = P.PRODUCT_ID;

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 					
	ORDER_ID	PRODUCT_NAME	QUANTITY	UNIT_PRICE	DISCOUNT_PCT
▶	1001	Wireless Earbuds	2	1499.00	0.00
	1006	Wireless Earbuds	1	1499.00	10.00
	1002	Cotton T-Shirt	1	799.00	0.00
	1011	Cotton T-Shirt	1	599.00	0.00
	1003	Smart Watch	1	2999.00	0.00
	1005	Smart Watch	1	2999.00	0.00
	1003	Running Shoes	1	4599.00	5.00
	1006	Running Shoes	1	4599.00	5.00
	1004	Bluetooth Speaker	1	3499.00	0.00
	1009	Bluetooth Speaker	1	3499.00	0.00
	1007	Bedsheet Set	1	1299.00	0.00
	1010	Bedsheet Set	1	1299.00	0.00
	1001	Laptop Stand	1	899.00	10.00
	1008	Laptop Stand	1	899.00	0.00
	1009	Cushion Covers (...)	2	599.00	15.00
	1010	Cushion Covers (...)	1	599.00	0.00
	1011	Cushion Covers (...)	1	999.00	0.00

-- Insight:

-- Gives detailed breakdown of each order's products.

-- Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

/*LEFT JOIN:

SELECT *

FROM customers c

LEFT JOIN orders o

ON c.customer_id = o.customer_id;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	order_date	status	total_amount
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	2024-08-01	Delivered	4498.00
	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101	2024-08-10	Delivered	3499.00
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102	2024-08-03	Delivered	799.00
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1011	102	2026-06-29	Pending	1598.00
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103	2024-08-05	Shipped	7498.00
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103	2024-08-20	Delivered	899.00
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	2024-08-12	Cancelled	2999.00
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105	2024-08-15	Delivered	5898.00
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0	1007	106	2024-08-18	Pending	1299.00
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1	1009	107	2024-08-25	Shipped	6098.00

1)All customers shown.

2)Orders only if exist.

RIGHT JOIN:

SELECT *

FROM customers c

RIGHT JOIN orders o

ON c.customer_id = o.customer_id;

customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	order_date	status	total_amount
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	2024-08-01	Delivered	4498.00
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102	2024-08-03	Delivered	799.00
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103	2024-08-05	Shipped	7498.00
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101	2024-08-10	Delivered	3499.00
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	2024-08-12	Cancelled	2999.00
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105	2024-08-15	Delivered	5898.00
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0	1007	106	2024-08-18	Pending	1299.00
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103	2024-08-20	Delivered	899.00
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1	1009	107	2024-08-25	Shipped	6098.00
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0	1010	108	2024-08-28	Delivered	1598.00
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1011	102	2026-06-29	Pending	1598.00

1)All orders shown

2)Customer only if exists

Explanation :-

LEFT JOIN returns all records from the left table and matching records from the right table.

RIGHT JOIN returns all records from the right table and matching records from the left table.

FULL OUTER JOIN

Returns:

All customers

All orders

Even if no match

Used when we want complete data from both tables

MySQL does not support FULL JOIN directly*/

-- Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

/*Foreign Key Relationships:

orders.customer_id -> customers.customer_id

order_items.order_id -> orders.order_id

order_items.product_id -> products.product_id

Explanation :-

Foreign Key:

1)Maintains relationship between tables

2)Ensures data consistency

If -

```
INSERT INTO orders (order_id, customer_id, order_date, status, total_amount)
```

```
VALUES (999, 999, '2024-08-10', 'Delivered', 5000);
```

(customer_id = 999 does NOT exist)

Result:

ERROR (Foreign Key Constraint Violation)

Explanation:

The database will reject the insertion because the referenced customer_id does not exist in the customers table. This ensures referential integrity.

*/

/*Section E — Advanced Concepts (CASE, ACID, Transactions)

These questions test your understanding of conditional logic, database reliability principles, and transaction management.

*/

/*Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier. */

SELECT PRODUCT_NAME, UNIT_PRICE,

CASE

 WHEN UNIT_PRICE <1000 THEN 'Budget'

 WHEN UNIT_PRICE BETWEEN 1000 AND 3000 THEN 'Mid-Range'

 ELSE 'Premium'

END AS price_tier

FROM PRODUCTS;

Result Grid   Filter Rows: Export: 			
	PRODUCT_NAME	UNIT_PRICE	price_tier
▶	Wireless Earbuds	1499.00	Mid-Range
	Cotton T-Shirt	799.00	Budget
	Smart Watch	2999.00	Mid-Range
	Running Shoes	4599.00	Premium
	Bluetooth Speaker	3499.00	Premium
	Bedsheet Set	1299.00	Mid-Range
	Laptop Stand	899.00	Budget
	Cushion Covers (Set)	599.00	Budget

-- Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

SELECT

COUNT(CASE WHEN STATUS = 'DELIVERED' THEN 1 END) AS DELIVERED_ORDERS,

COUNT(CASE WHEN STATUS <> 'DELIVERED' THEN 1 END) AS NOT_DELIVERED_ORDERS

FROM ORDERS;

Result Grid   Filter Rows: Export: 			
	DELIVERED_ORDERS	NOT_DELIVERED_ORDERS	
▶	6	5	

/*Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

ANS :-

A – Atomicity

Definition: Transaction is all or nothing

Example:

1000 transfer from A -> B

If money is deducted but not added -> No rollback

Ensures no partial transactions.

C – Consistency

Definition: Database must remain in a valid state

Example:

Total money before transfer = total after transfer

Rules or constraints are always maintained

I – Isolation

Definition: Transactions run independently

Example:

Two people transferring money simultaneously should not interfere

Prevents dirty reads / conflicts

D – Durability

Definition: Once committed, data is permanently saved

Example:

After transfer success, even if system crashes -> data remains

Ensured via logs / backups

*/

/*Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products
4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

Write the complete BEGIN...COMMIT/ROLLBACK block.*/

BEGIN;

INSERT INTO orders VALUES

(1011, 102, '2026-06-29', 'Pending', 1598.00);

INSERT INTO order_items VALUES

(5020, 1011, 208, 1, 999.00,0);

INSERT INTO order_items VALUES

(5021, 1011, 202, 1, 599.00,0);

UPDATE PRODUCTS

SET STOCK_QTY = STOCK_QTY - 1

WHERE PRODUCT_ID = 208;

UPDATE PRODUCTS

SET STOCK_QTY = STOCK_QTY - 1

WHERE PRODUCT_ID = 202;

COMMIT;

-- If any query fails, ROLLBACK should be executed manually

-- Extra Points to add.

-- Top Customer

SELECT

c.customer_id,

```
c.first_name,  
SUM(o.total_amount) AS total_spent  
FROM customers c  
JOIN orders o  
ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.first_name  
ORDER BY total_spent DESC  
LIMIT 1;  
  
-- Top Product  
SELECT  
p.product_name,  
SUM(oi.quantity) AS total_quantity  
FROM order_items oi  
JOIN products p  
ON oi.product_id = p.product_id  
GROUP BY p.product_name  
ORDER BY total_quantity DESC  
LIMIT 1;  
  
-- Validation  
SELECT email, COUNT(*)  
FROM customers  
GROUP BY email  
HAVING COUNT(*) > 1;
```